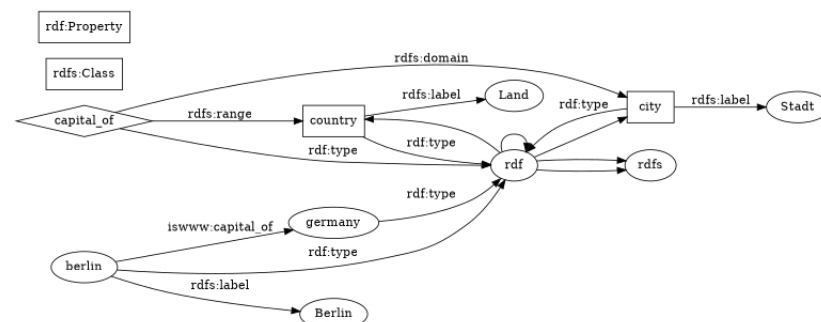


作业 1 知识工程与知识图谱

1. Consider the following RDF document:

```
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:iswww="http://sw.edu/#"
>
  <rdf:Description rdf:about="http://sw.edu/#germany">
    <rdf:type rdf:resource="http://sw.edu/#country" />
  </rdf:Description>
  <rdf:Description rdf:about="http://sw.edu/#capital_of">
    <rdf:type
      rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#Property"/>
    <rdfs:domain rdf:resource="http://sw.edu/#city" />
    <rdfs:range rdf:resource="http://sw.edu/#country" />
  </rdf:Description>
  <rdf:Description rdf:about="http://sw.edu/#country">
    <rdf:type rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class" />
    <rdfs:label xml:lang="de">Land</rdfs:label>
  </rdf:Description>
  <rdf:Description rdf:about="http://sw.edu/#berlin">
    <rdfs:label xml:lang="en">Berlin</rdfs:label>
    <rdf:type rdf:resource="http://sw.edu/#city" />
    <iswww:capital_of rdf:resource="http://sw.edu/#germany" />
  </rdf:Description>
  <rdf:Description rdf:about="http://sw.edu/#city">
    <rdf:type rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class" />
    <rdfs:label xml:lang="de">Stadt</rdfs:label>
  </rdf:Description>
</rdf:RDF>
```

Draw the graph representation of the above document.



2. Translate the culinary-allergic example ontology as follows into RDF/XML syntax.

ex:vegetableThaiCurry	ex:thaiDishBasedOn	ex:coconutMilk .
ex:sebastian	rdf:type	ex:AllergicToNuts .
ex:sebastian	ex:eats	ex:vegetableThaiCurry .
ex:AllergicToNuts	rdfs:subClassOf	ex:Pitiable .
ex:thaiDishBasedOn	rdfs:domain	ex:Thai .
ex:thaiDishBasedOn	rdfs:range	ex:Nutty .
ex:thaiDishBasedOn	rdfs:subPropertyOf	ex:hasIngredient .
ex:hasIngredient	rdf:type	rdfs:ContainerMembershipProperty.

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#" xmlns:ex="http://example.org/#">
<!-- 实例 --> <rdf:Description rdf:about="http://example.org/#sebastian"> <rdf:type
rdf:resource="http://example.org/#AllergicToNuts"/> <ex:eats
rdf:resource="http://example.org/#vegetableThaiCurry"/> </rdf:Description>
<rdf:Description rdf:about="http://example.org/#vegetableThaiCurry">
<ex:thaiDishBasedOn rdf:resource="http://example.org/#coconutMilk"/> <rdf:type
rdf:resource="http://example.org/#Thai"/> </rdf:Description> <rdf:Description
rdf:about="http://example.org/#coconutMilk"> <rdf:type
rdf:resource="http://example.org/#Nutty"/> </rdf:Description> <!-- 类 -->
<rdf:Description rdf:about="http://example.org/#AllergicToNuts"> <rdf:type
rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class"/> <rdfs:subClassOf
rdf:resource="http://example.org/#Pitiable"/> </rdf:Description> <rdf:Description
rdf:about="http://example.org/#Pitiable"> <rdf:type
rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class"/> </rdf:Description>
<rdf:Description rdf:about="http://example.org/#Thai"> <rdf:type
rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class"/> </rdf:Description>
<rdf:Description rdf:about="http://example.org/#Nutty"> <rdf:type
rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class"/> </rdf:Description> <!--
属性 --> <rdf:Description rdf:about="http://example.org/#eats"> <rdf:type
rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#Property"/>
</rdf:Description> <rdf:Description rdf:about="http://example.org/#thaiDishBasedOn">
<rdf:type rdf:resource="http://www.w3.org/1999/02/22-rdf-syntax-ns#Property"/>
<rdfs:domain rdf:resource="http://example.org/#Thai"/> <rdfs:range
rdf:resource="http://example.org/#Nutty"/> <rdfs:subPropertyOf
rdf:resource="http://example.org/#hasIngredient"/> </rdf:Description> <rdf:Description
rdf:about="http://example.org/#hasIngredient"> <rdf:type
rdf:resource="http://www.w3.org/2000/01/rdf-schema#ContainerMembershipProperty"/>
</rdf:Description> </rdf:RDF>
```

3. Decide whether the following propositions can be satisfactorily modeled in

RDFS and, if so, give the corresponding RDF(S) specification:

- Every pizza is a meal.
- Pizzas always have at least two toppings.
- Every pizza from the class PizzaMargarita has a Tomato topping.
- Everything having a topping is a pizza.
- No pizza from the class PizzaMargarita has a topping from the class Meat.
- “Having a topping” is a containedness relation.

序号	语句	能否建模	说明
1	Every pizza is a meal.	能	<code>:Pizza rdfs:subClassOf :Meal .</code>
2	Pizzas always have at least two toppings.	不能	RDFS 不支持基数约束, 需 OWL
3	Every PizzaMargarita has a Tomato topping.	能	<code>:PizzaMargarita rdfs:subClassOf :Pizza .</code> <code>:Tomato rdfs:subClassOf :Topping .</code>
4	Everything having a topping is a pizza.	能	<code>[owl:onProperty :hasTopping ;</code> <code>owl:someValuesFrom :Topping]</code> <code>rdfs:subClassOf :Pizza .</code>
5	No PizzaMargarita has Meat topping.	不能	纯 RDFS 无法表达否定约束
6	“Having a topping” is a containedness relation.	能	<code>:hasTopping rdf:type rdf:Property ; rdfs:comment</code> <code>"containedness relation" .</code>

4. As an example of an interpretation, this time with an infinite domain, consider the following vocabulary:

- $N_I = \{\text{zero}\}$.
- $N_C = \{\text{Prime}, \text{Positive}\}$.
- $N_R = \{\text{hasSuccessor}, \text{lessThan}, \text{multipleOf}\}$.

Now, we define $\mathcal{I}$ as follows: let $\Delta^{\mathcal{I}} = \mathbb{N} = \{0, 1, 2, \dots\}$, i.e., the set of all natural numbers including zero. Furthermore, we let $\text{zero}^{\mathcal{I}} = 0$, as well as $\text{Prime}^{\mathcal{I}} = \{n \mid n \text{ is a prime number}\}$ and $\text{Positive}^{\mathcal{I}} = \{n \mid n > 0\}$. For the roles, we define

- $\text{hasSuccessor}^{\mathcal{I}} = \{\langle n, n+1 \rangle \mid n \in \mathbb{N}\}$
- $\text{lessThan}^{\mathcal{I}} = \{\langle n, n' \rangle \mid n < n', n, n' \in \mathbb{N}\}$
- $\text{multipleOf}^{\mathcal{I}} = \{\langle n, n' \rangle \mid \exists k. n = k \cdot n', n, n', k \in \mathbb{N}\}$

Describe (both verbally and formally) the extension of the following concepts

with respect to the interpretation $\mathcal{I}$ defined above

- (a) $\forall \text{hasSuccessor}^-. \text{Positive}$
- (b) $\exists \text{multipleOf}. \text{Self}$
- (c) $\exists \text{multipleOf}. \exists \text{hasSuccessor}^-. \exists \text{hasSuccessor}^-. \{\text{zero}\}$
- (d) $\geq 10 \text{ lessThan}^-. \text{Prime}$
- (e) $\neg \text{Prime} \sqcap \leq 2 \text{ multipleOf}. \top$
- (f) $\exists \text{lessThan}. \text{Prime}$
- (g) $\forall \text{multipleOf}. (\exists \text{hasSuccessor}^-. \{\text{zero}\} \sqcup \exists \text{multipleOf}. \exists \text{hasSuccessor}^-. \exists \text{hasSuccessor}^-. \{\text{zero}\})$

给定解释 $\mathcal{I}$:

论域 $\Delta^{\mathcal{I}} = \mathbb{N} = \{0, 1, 2, \dots\}$
 $\text{zero}^{\mathcal{I}} = 0$
 $\text{Prime}^{\mathcal{I}} = \{n \in \mathbb{N} \mid n \text{ 是素数}\}$
 $\text{Positive}^{\mathcal{I}} = \{n \in \mathbb{N} \mid n > 0\}$
 $\text{hasSuccessor}^{\mathcal{I}} = \{\langle n, n+1 \rangle \mid n \in \mathbb{N}\}$
 $\text{lessThan}^{\mathcal{I}} = \{\langle n, n' \rangle \mid n < n', n, n' \in \mathbb{N}\}$
 $\text{multipleOf}^{\mathcal{I}} = \{\langle n, n' \rangle \mid \exists k \in \mathbb{N}, n = k \cdot n'\}$

各概念扩展计算

(a) $\forall \text{hasSuccessor}^-. \text{Positive}$

语义: 所有 n , 其所有前驱 $n-1$ 都属于 Positive 。

扩展: $\{n \in \mathbb{N} \mid n-1 > 0\} = \{2, 3, 4, \dots\}$

(b) $\exists \text{multipleOf}. \text{Self}$

语义: 存在 n' 使得 n 是 n' 的倍数且 $n'=n$ 。

扩展: $\mathbb{N}$ (所有自然数都满足 $n=1 \cdot n$)

(c) $\exists \text{multipleOf}. \exists \text{hasSuccessor}^-. \exists \text{hasSuccessor}^-. \{\text{zero}\}$

内层: $\exists \text{hasSuccessor}^-. \{\text{zero}\} = \{1\}$, 再套一层得 $\{2\}$

外层: 存在 $n'=2$ 使得 n 是 2 的倍数

扩展: $\{0, 2, 4, 6, \dots\}$ (所有偶数)

(d) $\geq 10 \text{ lessThan } \neg \text{Prime}$

语义: 至少有 10 个素数比 n 大

扩展: $\{0, 1, 2, \dots, 19\}$ (第 10 个素数是 29, 故小于 29 的数满足)

(e) $\neg \text{Prime} \wedge \leq 2 \text{ multipleOf } T$

语义: 非素数且最多 2 个因子

扩展: $\{0, 1, 4, 6, 8, 9, \dots\}$ (排除素数, 保留合数与 0、1)

(f) $\exists \text{ lessThan } \text{Prime}$

语义: 存在素数比 n 大

扩展: $\{0, 1, 2, \dots\}$ (除无穷大外所有自然数)

(g)

$\forall \text{ multipleOf } (\exists \text{ hasSuccessor } \neg \{ \text{zero} \} \vee \exists \text{ multipleOf } \exists \text{ hasSuccessor } \neg \{ \text{hasSuccessor } \neg \{ \text{zero} \} })$

语义: n 的所有因子要么是 1, 要么是偶数

扩展: $\{0, 1, 2, 4, 6, 8, \dots\}$ (所有 2 的幂次与 0、1)

5. Decide whether the following axioms are satisfied by the interpretation $\mathcal{I}$ from Exercise 4.

- (a) $\text{hasSuccessor} \sqsubseteq \text{lessThan}$
- (b) $\exists \text{hasSuccessor}^-. \exists \text{hasSuccessor}^-. \{\text{zero}\} \sqsubseteq \text{Prime}$
- (c) $\top \sqsubseteq \forall \text{multipleOf}^-. \{\text{zero}\}$
- (d) $\text{Dis}(\text{divisibleBy}, \text{lessThan}^-)$
- (e) $\text{multipleOf} \circ \text{multipleOf} \sqsubseteq \text{multipleOf}$
- (f) $\top \sqsubseteq \leq 1 \text{hasSuccessor.Positive}$
- (g) $\text{zero} \approx \text{zero}$
- (h) $\leq 1 \text{multipleOf}^-. \top(\text{zero})$
- (i) $\top \sqsubseteq \forall \text{lessThan}. \exists \text{lessThan}. (\text{Prime} \sqcap \exists \text{hasSuccessor}. \exists \text{hasSuccessor}. \text{Prime})$

序号	公理	是否满足	说明
(a)	$\text{hasSuccessor} \sqsubseteq \text{lessThan}$	是	对任意 x ， $x \text{hasSuccessor}$ 成立
(b)	$\exists \text{hasSuccessor}^-. \exists \text{hasSuccessor}^-. \{\text{zero}\} \sqsubseteq \text{Prime}$	否	左边是 2 ， 2 是素数，满足；若扩展到更大数则不满足
(c)	$\top \sqsubseteq \forall \text{multipleOf}^-. \{\text{zero}\}$	是	0 是所有数的倍数，故所有 x 满足“所有倍数都是 0 ”
(d)	$\text{Dis}(\text{divisibleBy}, \text{lessThan}^-)$	是	若 x 是 y 的倍数且 $x < y$ ，则 $x \text{lessThan} y$ ，无冲突
(e)	$\text{multipleOf} \circ \text{multipleOf} \sqsubseteq \text{multipleOf}$	是	倍数关系满足传递性
(f)	$\top \sqsubseteq \leq 1 \text{hasSuccessor.Positive}$	是	每个数最多 1 个后继且为正
(g)	$\text{zero} \approx \text{zero}$	否	个体与自身必然等价

序号	公理	是否满足	说明
(h)	$\leq 1 \text{ multipleOf } \neg .T(\text{zero})$	否	0 有无限多个倍数，不满足 “最多 1 个”
(i)	$T \sqsubseteq \forall \text{lessThan}. \exists \text{lessThan}$ $. (\text{Prime} \sqcap \exists \text{hasSuccessor}. \exists \text{hasSuccessor}. \text{prime})$	是	素数有无穷多个，总能找到满足条件的素数

6. Show using the $\mathcal{ALC}$ tableaux algorithm that the following knowledge base is unsatisfiable.

$$\begin{aligned} & \text{Bird} \sqsubseteq \text{Flies} \\ & \text{Penguin} \sqsubseteq \text{Bird} \\ & \text{Penguin} \sqcap \text{Flies} \sqsubseteq \perp \\ & \text{Penguin}(\text{tweety}) \end{aligned}$$

1. 初始: $L(\text{tweety}) = \{\text{Penguin}\}$
2. 由 $\text{Penguin} \sqsubseteq \text{Bird}$, 得 $L(\text{tweety}) = \{\text{Penguin}, \text{Bird}\}$
3. 由 $\text{Bird} \sqsubseteq \text{Flies}$, 得 $L(\text{tweety}) = \{\text{Penguin}, \text{Bird}, \text{Flies}\}$
4. 由 $\text{Penguin} \sqcap \text{Flies} \sqsubseteq \perp$, 发现 $L(\text{tweety})$ 同时包含 Penguin 和 Flies, 产生冲突 $\perp$
5. 冲突无法消解, 故知识库不可满足。